

Especialização em
IA & Machine Learning

IA Generativa & Engenharia de Prompts



Contexto do Encontro

Engenharia de Prompts e Pipelines

Durante muito tempo, a interação com modelos de IA Generativa foi vista apenas como uma conversa em linguagem natural. Entretanto, aplicações reais exigem muito mais do que perguntas bem formuladas.

Hoje, sistemas baseados em LLMs são capazes de:

- Interpretar documentos, consultar bases de conhecimento, executar ferramentas, raciocinar em múltiplas etapas, produzir respostas estruturadas, apoiar decisões complexas.

Para alcançar esse nível de confiabilidade, torna-se necessário projetar cuidadosamente a forma como os modelos recebem informações, raciocinam e produzem suas respostas.

Por que estudar Engenharia de Prompts?

- Um dos maiores equívocos no início da IA Generativa foi acreditar que escolher um modelo poderoso seria suficiente para construir uma boa solução.
- Na prática, o desempenho final depende muito mais da arquitetura construída ao redor do modelo do que do próprio modelo.
- **Pequenas mudanças no prompt podem gerar grandes diferenças na qualidade da resposta.**

O modelo não é o produto!!

O novo papel do Engenheiro de IA

- O profissional deixa de apenas consumir IA e passa a projetar sistemas inteligentes.
- A Engenharia de Prompts é apenas uma parte desse processo.
- O objetivo final é construir aplicações robustas, previsíveis e reproduzíveis.

O desafio da IA Generativa

O problema não é responder... mas responder corretamente!!

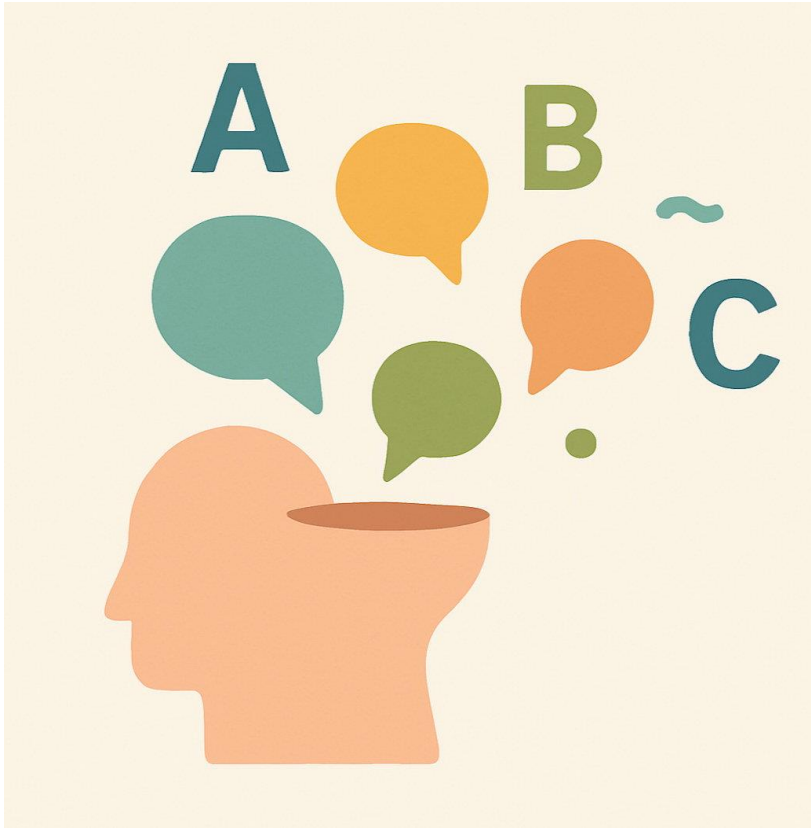
- Modelos de linguagem possuem enorme capacidade de geração de texto, porém também apresentam limitações importantes: **alucinações, respostas inconsistentes, perda de contexto, excesso de criatividade, dificuldade em problemas complexos, ausência de memória permanente.**
- Por isso, aplicações profissionais precisam incorporar mecanismos capazes de orientar o modelo durante todo o processo de geração.

Da conversa ao Pipeline

- Muitos problemas não podem ser resolvidos com uma única interação.
- **Um único prompt nem sempre é o suficiente!**

Exemplo:

"Analise este contrato, identifique riscos jurídicos e proponha melhorias."



Vocabulário Teórico

Vamos adiantar...

- *Zero-Shot, Few-Shot, Prompt Chaining, Self-Ask, Tree of Thoughts, ReAct, LangChain, Pydantic, Whisper, Groq...*



Conteúdo Programático

Ementa

Primeiro, vamos entender os conceitos:

- Prompts Estruturados
- *Zero-Shot, Few-Shot*
- *Prompt Chaining*
- *Self-Ask*
- *Tree of Thoughts*
- *ReAct*
- Saídas Estruturadas
- Validação
- Avaliação

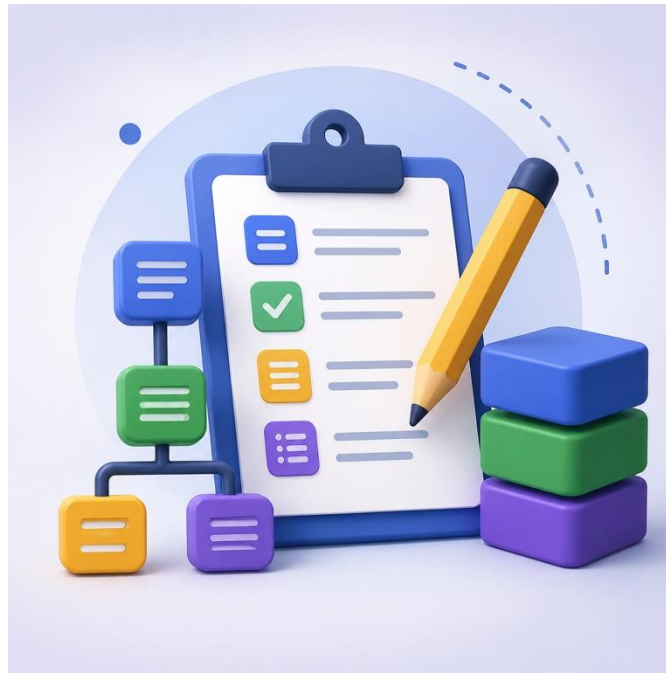
Ementa

Agora, vamos conhecer algumas ferramentas:

- *UV, LangChain, Groq API, Whisper, Pandas AI, Pydantic*, outros.

E trabalhar um pouco:

- Consumo de modelos externos
- Criação de *Pipelines*
- Assistente Conversacional



Prompts Estruturados

O que é um Prompt Estruturado?

- Um **Prompt Estruturado** é uma instrução organizada em componentes bem definidos, criada para orientar o modelo de linguagem a compreender corretamente o contexto, o objetivo da tarefa e o formato esperado da resposta.
- Ao contrário de perguntas abertas, um prompt estruturado reduz ambiguidades e aumenta a previsibilidade das respostas.

Um prompt estruturado normalmente contém:

- **Contexto** → Qual é o cenário?
- **Objetivo** → O que deve ser realizado?
- **Instruções** → Como executar a tarefa?
- **Restrições** → O que deve ou não deve fazer?
- **Formato de saída** → Como apresentar a resposta?

Quando utilizar?

- Quanto maior a clareza das instruções, maior tende a ser a qualidade da resposta produzida.
- Funciona como um contrato entre o usuário e o modelo de linguagem. Quanto mais explícito for esse contrato, menor será a necessidade de "adivinhar" a intenção do usuário e maior será a qualidade.

Aplicações Comuns

1. Gerar documentos.
2. Produzir relatórios.
3. Resumir textos.
4. Analisar contratos.
5. Criar planos de aula.

Benefícios

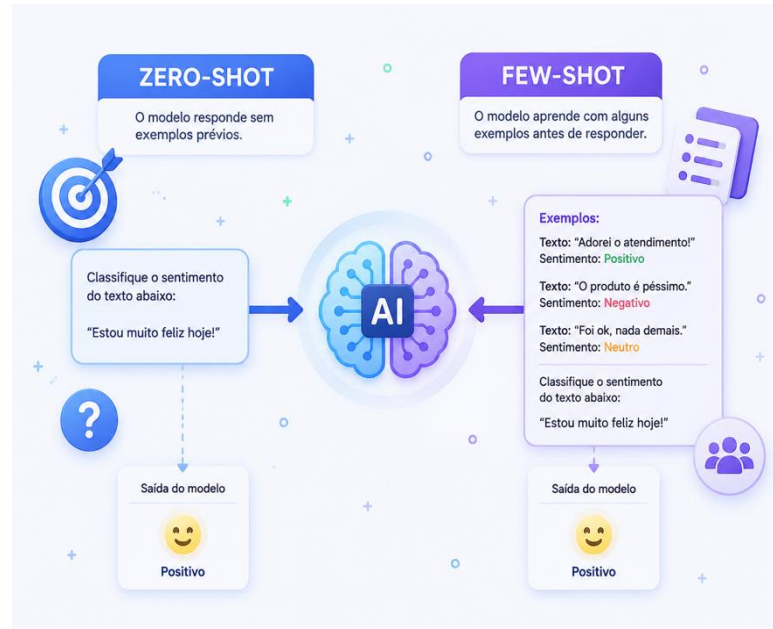
1. Redução de ambiguidades.
2. Consistência entre execuções.
3. Melhor entendimento do contexto.

Boas Práticas

1. Defina claramente o papel do modelo.
2. Explique o contexto da tarefa.
3. Informe restrições.
4. Formato de resposta.



Vamos Praticar



Zero-Shot, Few-Shot

Zero-Shot Prompting

- O modelo recebe apenas a instrução da tarefa, sem qualquer exemplo de como ela deve ser executada.
- O LLM utiliza exclusivamente o conhecimento adquirido durante seu treinamento para interpretar a solicitação e produzir uma resposta.
- **Características:** não utiliza exemplos, simples e rápido de construir, depende da capacidade geral do modelo, ideal para tarefas comuns ou bem definidas.

Few-Shot Prompting

- Além da instrução, são fornecidos alguns exemplos de entrada e saída. Esses exemplos funcionam como um guia para que o modelo compreenda melhor o padrão esperado.
- Em vez de aprender novos conhecimentos, o modelo aprende **como responder** àquela tarefa específica observando poucos exemplos.

Comparações

Zero-Shot

Apenas instruções

Mais simples

Respostas podem variar

Ideal para tarefas conhecidas

Few-Shot

Instruções + exemplos

Mais orientado

Maior consistência

Ideal para tarefas específicas ou padronizadas

Insight

*“**Zero-Shot** avalia a capacidade natural do modelo de resolver uma tarefa apenas com instruções. **Few-Shot**, por sua vez, utiliza exemplos como mecanismo de orientação, reduzindo ambiguidades e aumentando a consistência das respostas sem necessidade de retreinamento do modelo.”*



Prompt Chaining

O que é?

- ***Prompt Chaining*** é uma técnica em que uma tarefa complexa é dividida em várias etapas menores, cada uma executada por um prompt específico.
- A saída de uma etapa torna-se a entrada da etapa seguinte, formando um fluxo lógico de processamento.
- Em vez de solicitar ao modelo que resolva um problema complexo de uma única vez, o processo é organizado em uma sequência de tarefas menores, mais simples e mais controladas.

Benefícios: redução da complexidade, maior previsibilidade das respostas, melhor controle sobre cada etapa, facilidade de identificar erros, reutilização de etapas em diferentes fluxos.

Onde podemos encontrar?

- Em assistentes virtuais, chatbots corporativos, agentes inteligentes, geração automática de documentos, revisão de contratos, fluxos de atendimentos.
- Alguns *frameworks*: *LangChain*, *LangGraph*, *LlamaIndex*, *Semantic Kernel*, *CrewAI*, *AutoGen*.

Representa a transição entre "*fazer uma pergunta ao modelo*" e "*projetar um fluxo inteligente de resolução de problemas*".



Vamos Praticar



Self-Ask

O que é?

- **Self-Ask** é uma estratégia de *prompting* em que o modelo decompõe automaticamente um problema complexo em perguntas menores antes de produzir a resposta final.
- Em vez de responder imediatamente, o modelo conduz um processo interno de investigação, formulando perguntas intermediárias que o ajudam a construir um raciocínio mais consistente.
- Essa abordagem melhora a qualidade das respostas em tarefas que exigem múltiplas etapas de análise.

Exemplo

- **Pergunta:** *Qual tecnologia seria mais adequada para desenvolver uma plataforma de streaming para milhões de usuários?*

Antes de responder, faça perguntas intermediárias para analisar o problema.

Considere aspectos como:

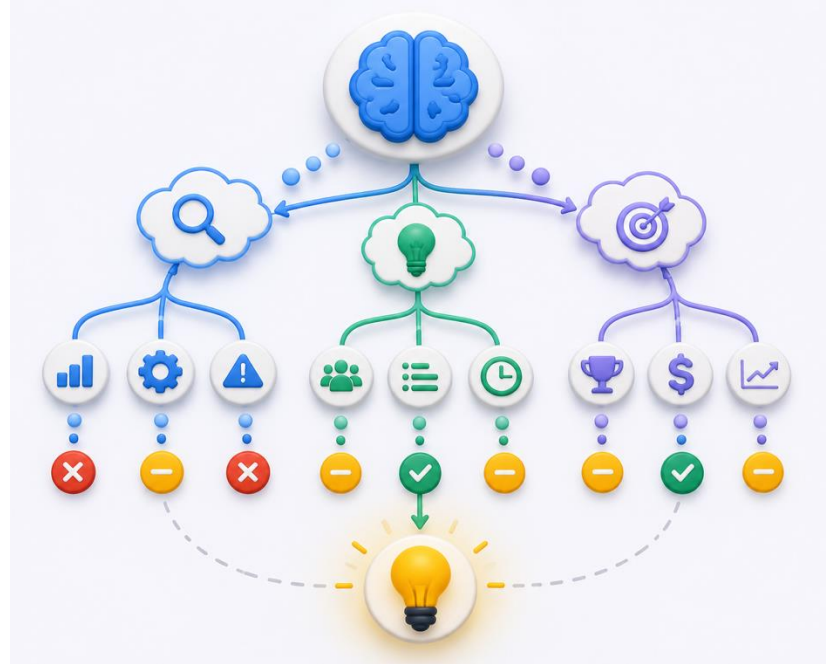
- volume de usuários;
- escalabilidade;
- armazenamento;
- distribuição de conteúdo;
- disponibilidade;
- custos.

Depois apresente a recomendação final.

Insight

*“Enquanto o **Prompt Chaining** divide o problema em várias chamadas ao modelo, o **Self-Ask** incentiva o próprio modelo a dividir o problema em perguntas intermediárias dentro de uma única interação.*

Em outras palavras, o encadeamento das etapas passa a fazer parte do raciocínio do LLM.”



Tree of Thoughts

TREE OF THOUGHTS (ToT)

Explorando múltiplos caminhos de raciocínio para encontrar a melhor solução

O QUE É?

O Tree of Thoughts (ToT) é uma técnica de raciocínio que permite ao modelo de IA explorar múltiplos caminhos possíveis para resolver um problema antes de escolher a melhor solução. Diferente do Chain of Thought (CoT), que segue um único caminho linear, o ToT cria uma árvore de possibilidades, avalia cada uma e expande apenas as mais promissoras.

COMPARAÇÃO: CoT vs ToT

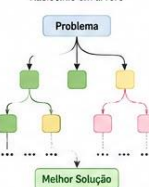
CHAIN OF THOUGHT (CoT)

Raciocínio linear



TREE OF THOUGHTS (ToT)

Raciocínio em árvore



■ Promissor
 ■ Em avaliação
 ■ Descartado
 ■ Explorado

CASOS DE USO REAIS



Arquitetura de Software

Escolha entre monolito, microsserviços, serverless, etc.



Desenvolvimento de Software

Definição de frameworks, patterns, banco de dados, autenticação, etc.



Planejamento de Projetos

Definição de escopo, recursos, cronograma e riscos.



Finanças

Escolha de investimentos, análise de risco, alocação de recursos.



Marketing

Canais, campanhas, público-alvo, análise de ROI.



Tomada de Decisão Multicritério

Decisões que envolvem vários fatores e cenários.



IA e Agentes

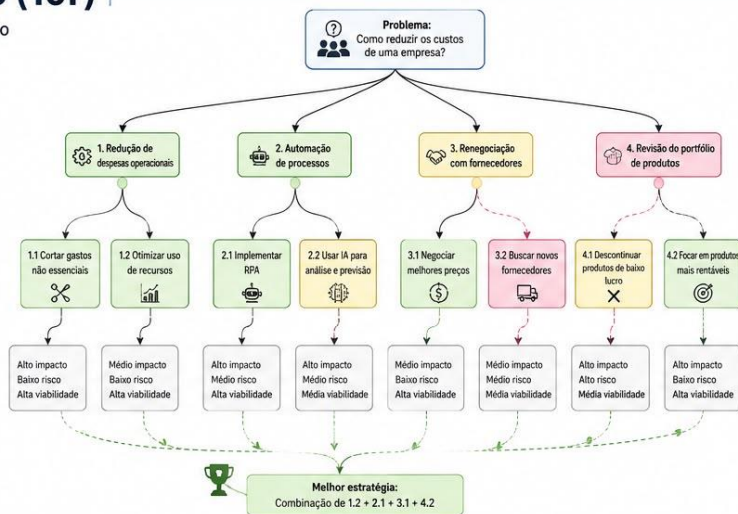
Planejamento de ações, estratégias de agentes, exploração de ferramentas.



Escrita Criativa

Exploração de ideias, finalizações e estilos diferentes.

EXEMPLO: "Como reduzir os custos de uma empresa?"



COMO FUNCIONA? (5 ETAPAS)

- Geração de pensamentos**
O modelo gera múltiplas possibilidades (ideias ou próximos passos).
- Avaliação**
Cada alternativa é avaliada com base em critérios definidos (impacto, risco, viabilidade, etc.).
- Expansão**
Apenas os caminhos mais promissores são expandidos com novos pensamentos.
- Backtracking**
Caminhos que se mostram fracos são descartados e o modelo retorna para explorar outras opções.
- Seleção da melhor solução**
Após explorar e comparar os caminhos, o modelo escolhe a melhor solução ou combinação.

LEGENDA

- Promissor: caminho com alta avaliação
- Em avaliação: ainda está sendo analisado
- Descartado: caminho pouco promissor
- Explorado: já avaliado

FORMAS DE IMPLEMENTAÇÃO

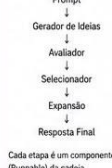
PROMPT ENGINEERING (Manual)

Passo a passo no prompt:

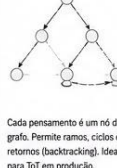
1. Gere várias ideias.
2. Avalie cada uma.
3. Escolha as melhores.
4. Expanda cada uma.
5. Escolha a vencedora.
6. Justifique sua decisão.

Simple, mas menos escalável.

2. COM LANCHAIN



3. COM LANGGRAPH



VANTAGENS E LIMITAÇÕES

VANTAGENS

- ✓ Explora múltiplas estratégias.
- ✓ Melhora a qualidade das respostas.
- ✓ Reduz risco de seguir caminho errado.
- ✓ Mais próximo do raciocínio humano.
- ✓ Ótimo para problemas complexos.

LIMITAÇÕES

- ✗ Alto custo de tokens e tempo.
- ✗ Muitas chamadas ao modelo.
- ✗ Excesso para problemas simples.
- ✗ Requer critérios claros de avaliação.

QUANDO USAR?

- ✓ Planejamento estratégico
 - ✓ Arquitetura de software
 - ✓ Desenvolvimento complexo
 - ✓ Tomada de decisão multicritério
 - ✓ Agentes de IA
 - ✓ Problemas de lógica e matemática
 - ✓ Cenários com muitas alternativas
- ✗ Não indicado para tarefas simples como: tradução, resumo, perguntas diretas.



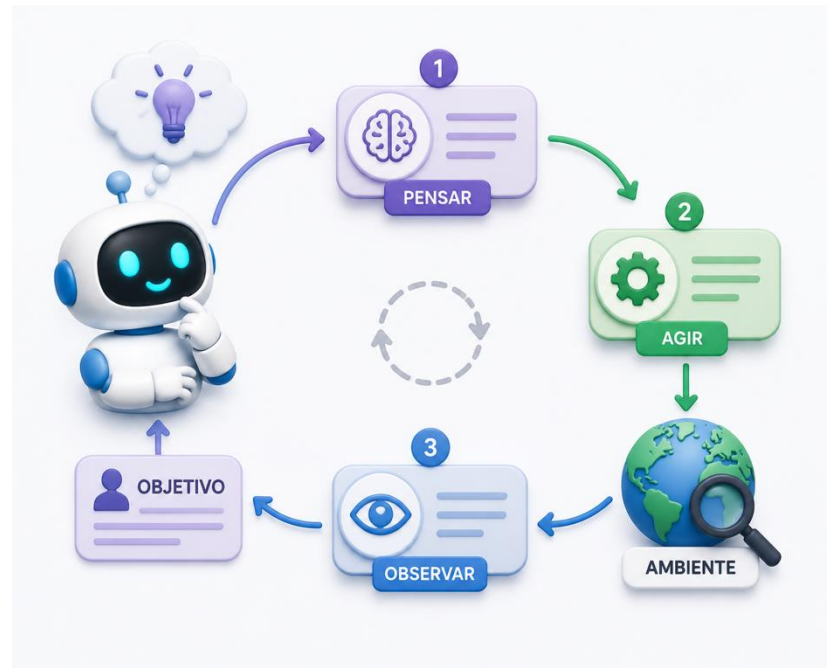
RELAÇÃO COM LANGGRAPH

O LangGraph materializa naturalmente o conceito do Tree of Thoughts. Ele permite criar grafos com ramificações, ciclos, avaliação de estados e retorno a etapas anteriores, representando o processo de exploração, avaliação e seleção que caracteriza o ToT.

REFERÊNCIAS

- Yao, S., Yu, D., Zhao, J., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv: 2305.10601.
- <https://reameprompting.org/docs/advanced/decomposition/tree-of-thoughts>
- <https://www.zm.com/think/topics/tree-of-thoughts>
- Documentação LangChain: <https://python.langchain.com/>
- Documentação LangGraph: <https://langchain-ai.github.io/langgraph/>





ReAct

ReAct (REASON + ACT)

Racionar e Agir para Resolver Problemas com LLMs

O QUE É?

ReAct (Reason + Act) é uma técnica que combina raciocínio (Reasoning) e execução de ações (Acting) permitindo que o modelo interaja com o ambiente usando ferramentas e iterando até chegar à resposta final.

COMO FUNCIONA?

- O modelo **RECEBE** uma pergunta ou objetivo.
- Ele **PENSA** sobre o que precisa fazer.
- Ele **ESCOLHE** uma ferramenta apropriada.
- EXECUTA** a ação e observa o resultado.
- Com a nova informação, ele **PENSA** novamente.
- O ciclo se repete até ter informação suficiente.
- Ele **FORNECE** a resposta final.

COMPARAÇÃO

	Chain of Thought (CoT)	Tree of Thoughts (ToT)	ReAct (Reason + Act)
Objetivo	Raciocínio linear passo a passo	Explorar múltiplos caminhos de raciocínio	Racionar e agir usando ferramentas
Interage com Ambiente?	Não	Não	Sim
Explora múltiplos caminhos?	Não	Sim	Não (um caminho por vez)
Usa Ferramentas?	Não	Não	Sim
Melhor para	Problemas lógicos e explicações	Planejamento e tomada de decisão complexa	Agentes, RAG, APIs, bancos, ações no mundo real

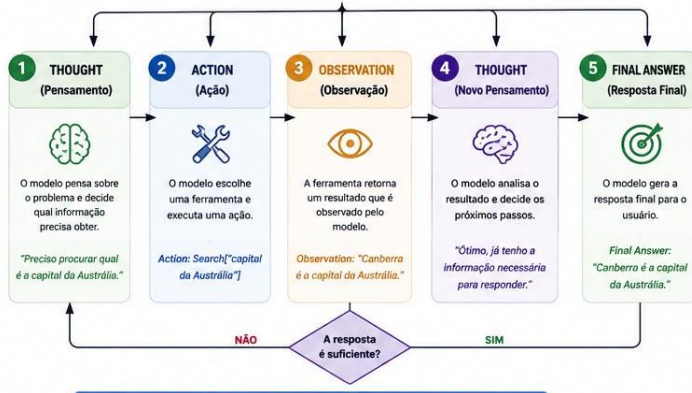
VANTAGENS

- ✓ Acesso a informações atualizadas.
- ✓ Reduz alucinações ao consultar fontes reais.
- ✓ Permite tarefas complexas passo a passo.
- ✓ Integração com várias ferramentas e sistemas.
- ✓ Ideal para agentes autônomos.

LIMITAÇÕES

- ✗ Mais caro (mais chamadas ao modelo/ferramentas).
- ✗ Maior latência dependendo das ações.
- ✗ Risco de loops infinitos (precisa de controle).
- ✗ Depende da qualidade das ferramentas.
- ✗ Requer bom design de prompts e ferramentas.

O CICLO ReAct



EXEMPLO COMPLETO

Pergunta do usuário: "Quem criou o Python e em que ano?"

Thought 1: Preciso pesquisar informações confiáveis sobre o criador do Python e o ano de criação.

Action 1: Search["criador da linguagem Python"]

Observation 1: Python foi criado por Guido van Rossum.

Thought 2: Agora preciso descobrir em que ano o Python foi criado.

Action 2: Search["ano de criação da linguagem Python"]

Observation 2: Python foi lançado pela primeira vez em 1991.

Final Answer: Python foi criado por Guido van Rossum e lançado em 1991.

QUANDO USAR REACT?

Use ReAct quando o problema exigir:

- Buscar Informações
- Consultar Dados
- Executar Ações
- Integrar com Sistemas
- Agentes Autônomos

FERRAMENTAS QUE PODEM SER USADAS

- Pesquisa Web
- Base de Dados (SQL)
- APIs
- RAG / Vector Store
- Calculadora
- Leitura/Escrita de Arquivos
- Execução de Código
- Sistemas Internos
- Outras Ferramentas

CASOS DE USO REAIS

- Assistentes Inteligentes que usam APIs e serviços.
- Consulta a bases de dados e geração de relatórios.
- Análise de código, execução de testes e debugging.
- Suporte técnico com acesso à documentação.
- Agentes de DevOps e monitoramento de sistemas.
- Pesquisa e análise de informações em tempo real.

COMO IMPLEMENTAR?

1. PROMPT ENGINEERING (Padrão ReAct)

- Usar um prompt que oriente o modelo a seguir o ciclo Thought → Action → Observation.
- Exemplo:
Thought: ...
Action: ...
Observation: ...
... (repetir)
Final Answer: ...

2. COM LANGCHAIN

- Usar agentes ReAct, AgentExecutor e Tools.
- Ferramentas são registradas e o agente decide quando usá-las.
- Fácil integração com LLMs, memória, RAG, databases, APIs, etc.

3. COM LANGGRAPH

- Modelar cada etapa como nós do grafo: Thought → Action → Observation → ...
- Permite ciclos, decisões condicionais e estado persistente.
- Ideal para agentes complexos e duráveis.

REFERÊNCIA

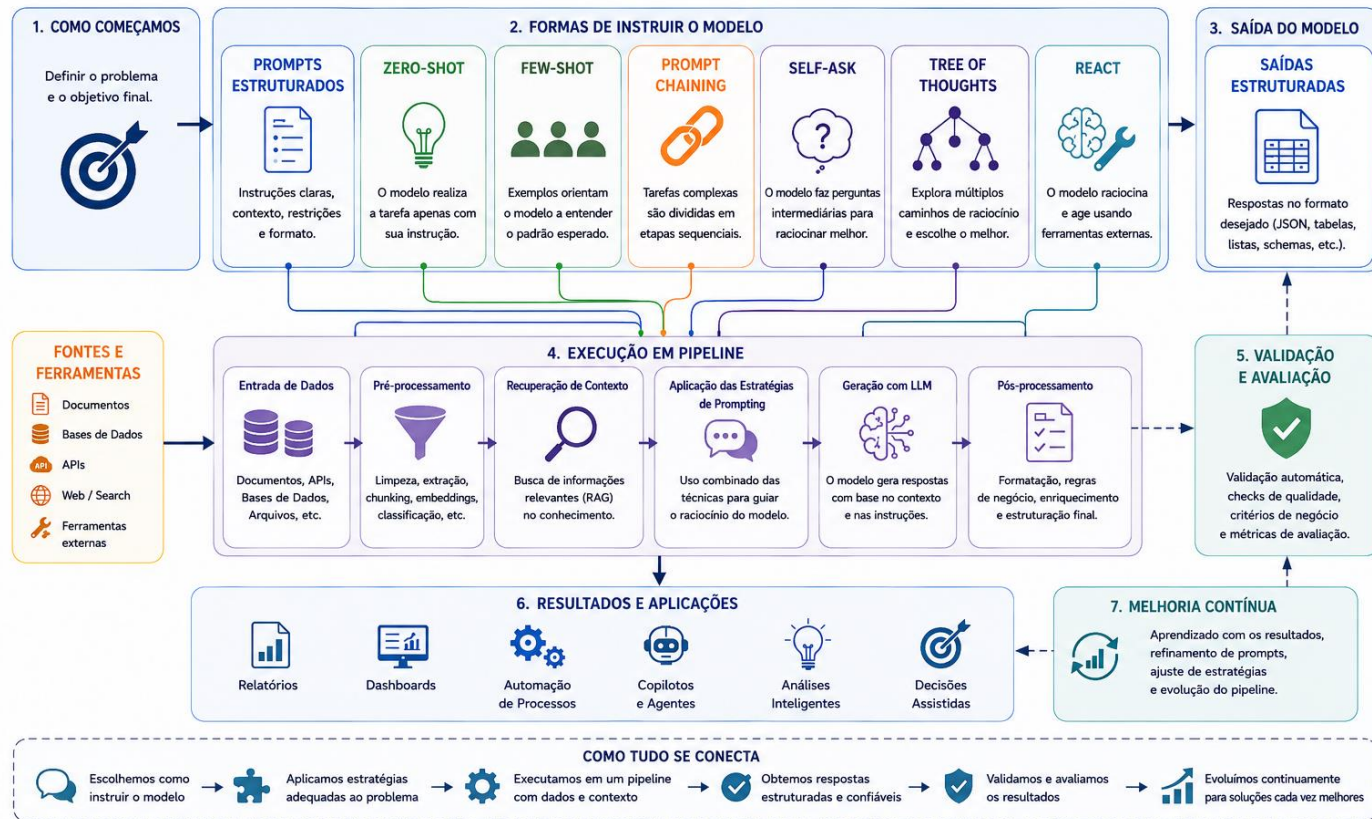
Artigo Original:
Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv: 2210.03629

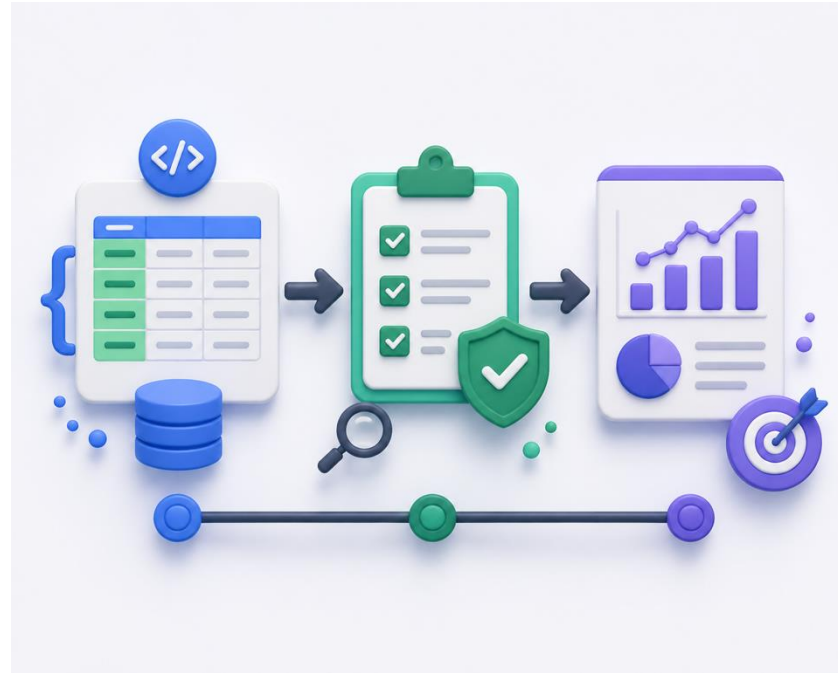
GitHub: <https://github.com/vaymyst/ReAct>
LangChain Docs: https://python.langchain.com/docs/modules/agents/agent_types/react
LangGraph Docs: <https://langchain-ai.github.io/langgraph/>



ENGENHARIA DE PROMPTS E PIPELINES INTELIGENTES

Da instrução à solução confiável





Saídas Estruturadas, Validação, Avaliação

Saídas Estruturadas

- Consiste em orientar o LLM para produzir respostas em um formato previamente definido, facilitando a leitura por pessoas e por sistemas.
- Formatos comuns: JSON, XML, *Markdown*, CSV, Tabelas, outros...

Exemplo

Prompt

text

Extraia as informações abaixo no formato JSON.

Nome: João Silva

Idade: 32

Cidade: São Paulo

Saída

json

```
{  
  "nome": "João Silva",  
  "idade": 32,  
  "cidade": "São Paulo"  
}
```

Objetivo: tornar a resposta previsível e fácil de integrar com aplicações.

Validação

- Processo de verificar se a resposta produzida pelo modelo atende aos requisitos definidos.
- O que pode ser validado: formato correto, campos obrigatórios, regras de negócio, consistência, limites de tamanho, valores permitidos.

Exemplo

Prompt pede:

```
json  
  
{  
  "nome": "",  
  "idade": 0  
}
```

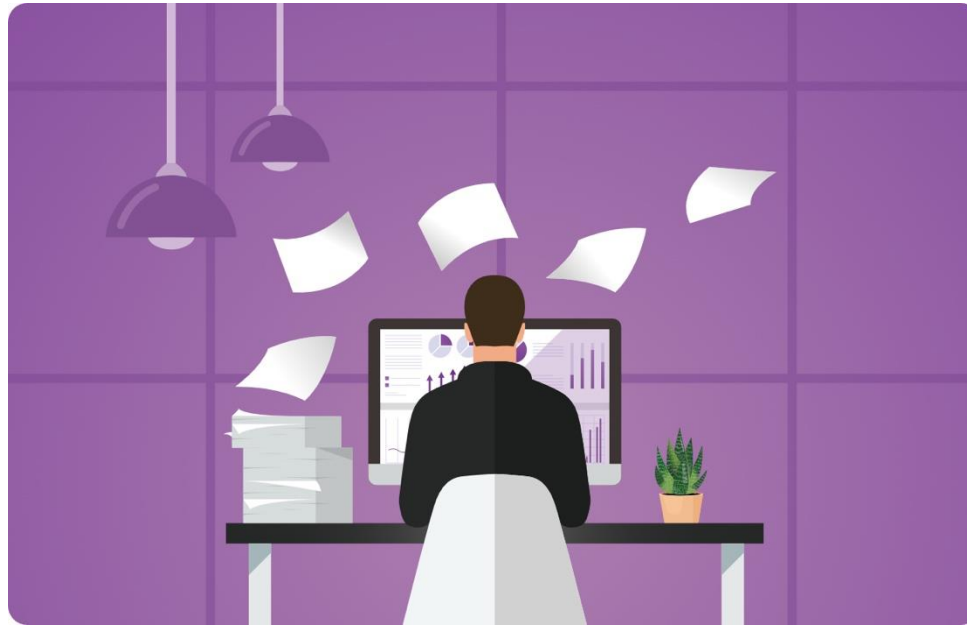
Validação verifica:

- ✓ JSON válido
- ✓ Idade numérica
- ✓ Nome preenchido

Objetivo: evitar respostas incorretas ou inconsistentes.

Avaliação

- Consiste em medir a qualidade das respostas geradas pelo modelo.
- Critérios comuns: precisão, clareza, completude, relevância, consistência, tempo de resposta.
- Formas de avaliação: avaliação humana, avaliação por outro LLM (*LLM as a Judge*), métricas automáticas.
- **Objetivo:** medir continuamente a qualidade do sistema e identificar oportunidades de melhoria.



Laboratórios

Só vamos!

- Conhecendo o Groq Cloud.
- Criando *pipelines* com o LangChain.
- Desenvolvendo nosso primeiro assistente conversacional, com análise de documentos.



Facens

AQUI TEM ENGENHARIA